

Experiment 9

Aim: Develop a mobile application that interacts with a RESTful API to fetch and display data from a remote server.

Theory:

RESTful APIs allow mobile applications to communicate with remote servers and exchange data over the internet using HTTP methods such as GET, POST, PUT, and DELETE. They are commonly used to fetch dynamic content and display real-time information in applications.

In this experiment, a Flutter application is developed to fetch and display quotes from a remote server using a REST API. The `http` package is used to send HTTP GET requests, while JSON data received from the server is parsed and converted into Dart objects.

The application uses `FutureBuilder` to handle asynchronous data loading and display the fetched quotes dynamically in a list view. A refresh feature is also implemented to reload data from the server. This experiment demonstrates API integration, JSON parsing, asynchronous programming, and real-time data handling in Flutter applications.

Source Code:

post.dart:

```
class Post {
  final int id;
  final String title;
  final String body;

  Post({required this.id, required this.title, required
this.body});

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'],
      title: json['title'],
      body: json['body'],
    );
  }
}
```

main.dart :

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'dart:ui';

void main() {
```

```
runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: QuoteScreen(),
    );
  }
}

class Quote {
  final String content;
  final String author;

  Quote({required this.content, required this.author});

  factory Quote.fromJson(Map<String, dynamic> json) {
    return Quote(
      content: json['quote'],
      author: json['author'],
    );
  }
}

class QuoteScreen extends StatefulWidget {
  @override
  _QuoteScreenState createState() => _QuoteScreenState();
}

class _QuoteScreenState extends State<QuoteScreen> {
  late Future<List<Quote>> quotes;

  Future<List<Quote>> fetchQuotes() async {
    final response = await
http.get(Uri.parse('https://dummyjson.com/quotes'));
  }
}
```

```
    if (response.statusCode == 200) {
        final data = json.decode(response.body);
        List results = data['quotes'];
        return results.map((e) => Quote.fromJson(e)).toList();
    } else {
        throw Exception("Failed to load quotes");
    }
}

@override
void initState() {
    super.initState();
    quotes = fetchQuotes();
}

Future<void> refreshQuotes() async {
    setState(() {
        quotes = fetchQuotes();
    });
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        extendBodyBehindAppBar: true,
        appBar: AppBar(
            title:
                Text("Daily Quotes", style: TextStyle(fontWeight:
FontWeight.bold)),
            centerTitle: true,
            backgroundColor: Color(0xffffefcfc),
            elevation: 0,
        ),
        body: Container(
            decoration: BoxDecoration(
                gradient: LinearGradient(
                    colors: [
```

```
        Color(0xFF1D2B64),
        Color(0xFFF8CDDA),
    ],
    begin: Alignment.topLeft,
    end: Alignment.bottomRight,
),
),
child: FutureBuilder<List<Quote>>(
    future: quotes,
    builder: (context, snapshot) {
        if (snapshot.connectionState ==
ConnectionState.waiting) {
            return Center(
                child: CircularProgressIndicator(color:
Colors.white),
            );
        }

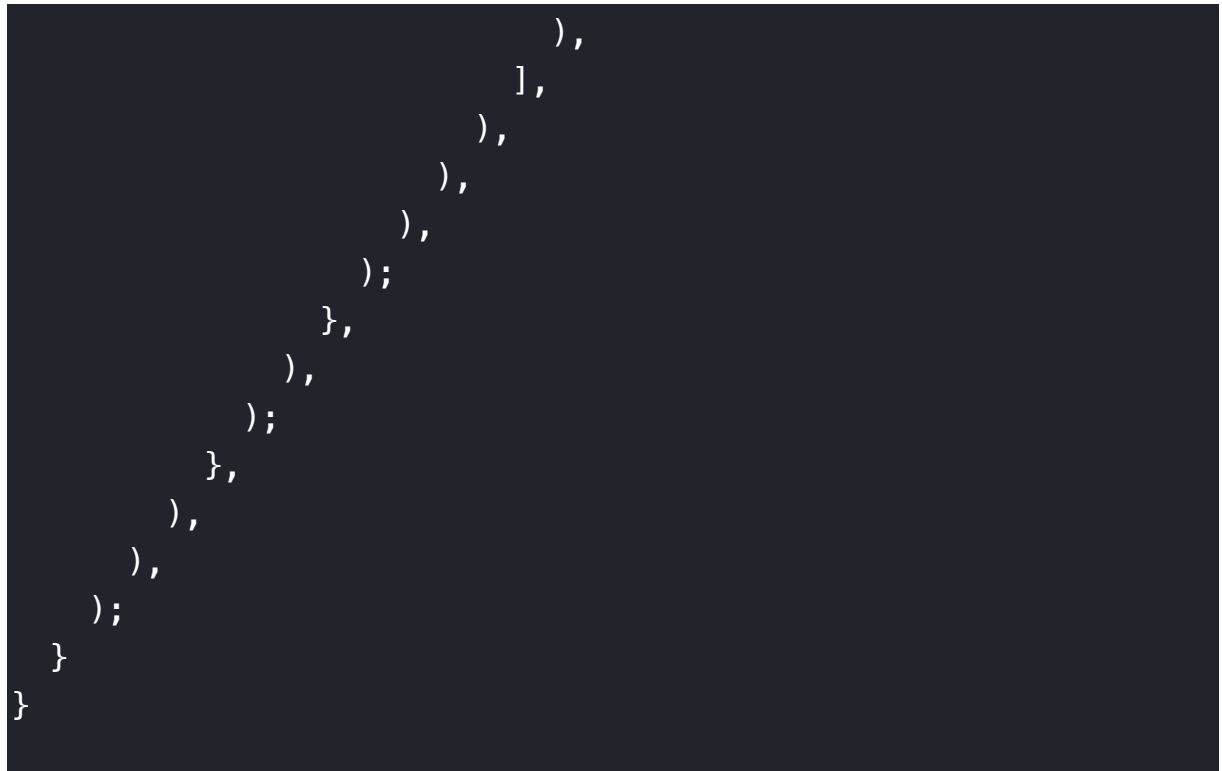
        if (snapshot.hasError) {
            return Center(
                child: Text(
                    "Error: ${snapshot.error}",
                    style: TextStyle(color: Colors.white),
                ),
            );
        }

        final data = snapshot.data!;

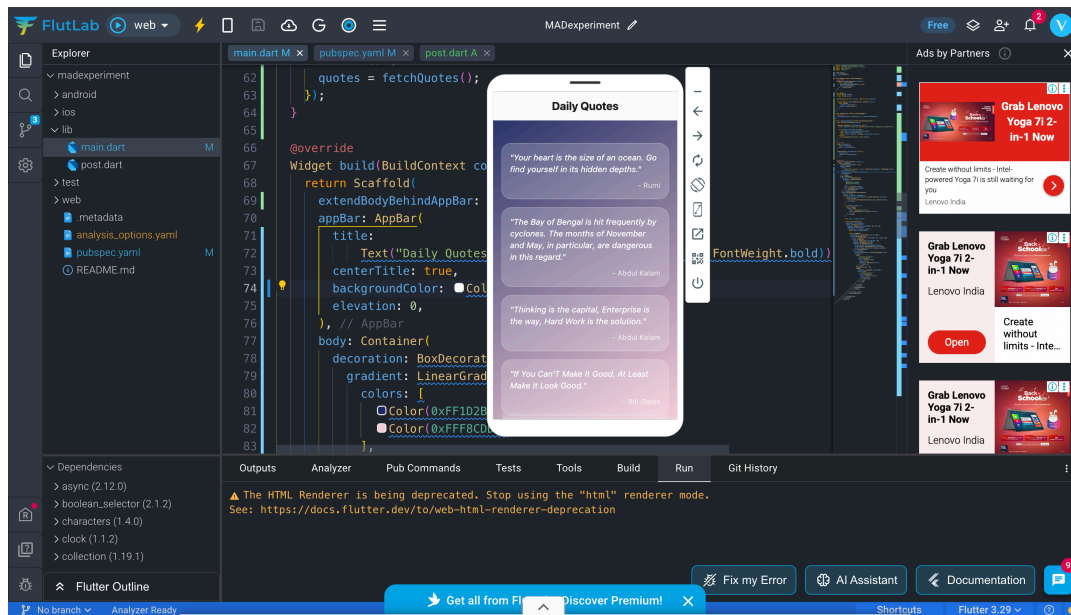
        return RefreshIndicator(
            onRefresh: refreshQuotes,
            child: ListView.builder(
                padding: EdgeInsets.fromLTRB(16, 100, 16, 16),
                itemCount: data.length,
                itemBuilder: (context, index) {
                    final quote = data[index];

                    return ClipRRect(
```

```
borderRadius: BorderRadius.circular(20),
child: BackdropFilter(
  filter: ImageFilter.blur(sigmaX: 10,
sigmaY: 10),
  child: Container(
    margin: EdgeInsets.only(bottom: 16),
    padding: EdgeInsets.all(16),
    decoration: BoxDecoration(
      color:
Colors.white.withOpacity(0.2),
      borderRadius:
BorderRadius.circular(20),
      border: Border.all(color:
Colors.white30),
    ),
    child: Column(
      crossAxisAlignment:
CrossAxisAlignment.start,
      children: [
        Text(
          "\"${quote.content}\"",
          style: TextStyle(
            color: Colors.white,
            fontSize: 16,
            fontStyle: FontStyle.italic,
          ),
        ),
        SizedBox(height: 10),
        Align(
          alignment:
Alignment.centerRight,
          child: Text(
            "- ${quote.author}",
            style: TextStyle(
              color: Colors.white70,
              fontSize: 14,
            ),
          ),
        ),
      ],
    ),
  ),
),
```



Output:



Learning Outcomes: